Next you will add some code to the layout page that will display the authenticated user's name or a link to the login page if they are not authenticated. Place the code block in listing 9.6 just after the `div` containing the `ul` element that houses the navigation links in the `nav` element.

Listing 9.6 Displaying the user's name or a link to sign in

```
<div class="align-content-end">
    @if (User.Identity.IsAuthenticated)                    ❶
    {
        <p>@User.Identity.Name, you are logged in</p>      ❷
    }
    else
    {
        <a asp-page="/login">Sign in</a>
    }
</div>
```

❶ Check to see if the user is authenticated.

❷ Use the User.Identity.Name property to access the user's name.

The `ClaimsPrincipal` is accessed via the page's `User` property. Its `Identity` property provides access to the primary claims identity for the user. It exposes two useful properties: `IsAuthenticated`, to enable you to establish whether the current request is from an authenticated user or an anonymous one, and `Name`, which gets the value assigned to the `Name` claim.

There are two more steps to take. The first is to force the user to sign in. You will do that by issuing a *challenge*, using an action result I didn't cover in chapter 3: the `ChallengeResult`. When used with cookies, a challenge redirects the anonymous user to the configured `loginpath`, which is /Account /Login by default. You are working with a different path, so you need to configure that first. Navigate to Program.cs, and amend the authentication service registration to include cookie options as follows.

Listing 9.7 Configuring the default login path

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults
➥ .AuthenticationScheme)
```